

**INSTITUTO
FEDERAL**

Santa Catarina

Câmpus
São José

FSM Configuração Relógio

AI2-2

ELD129003 - ELETRÔNICA DIGITAL II (2026.1)

Professor: Roberto de Matos

Aluno: Wagner Flores dos Santos

Junho de 2026

Sumário

1. Introdução	3
2. Descrição da Implementação	3
2.1. Estrutura da FSM	4
3. Processo de Desenvolvimento	4
4. Diagrama da Máquina de Estados	5
4.1. Implementação da FSM em VHDL	5
4.2. Modificações no arquivo top_timer_de2_115.vhd	9
5. Testes Realizados	11
6. RTL Viewer	18
7. Conclusão	19

1. Introdução

Este trabalho apresenta o desenvolvimento de uma Máquina de Estados Finitos (FSM) para controle da configuração de um relógio digital implementado em VHDL para a plataforma DE2-115.

A FSM permite ao usuário configurar horas, minutos e segundos através dos botões da placa, utilizando um conjunto de estados responsáveis pela navegação, incremento e decremento dos valores configurados.

2. Descrição da Implementação

O projeto foi desenvolvido a partir do arquivo base fornecido pelo professor, sendo realizada a implementação da lógica de configuração do relógio através de uma Máquina de Estados Finitos (FSM).

Os sinais utilizados são:

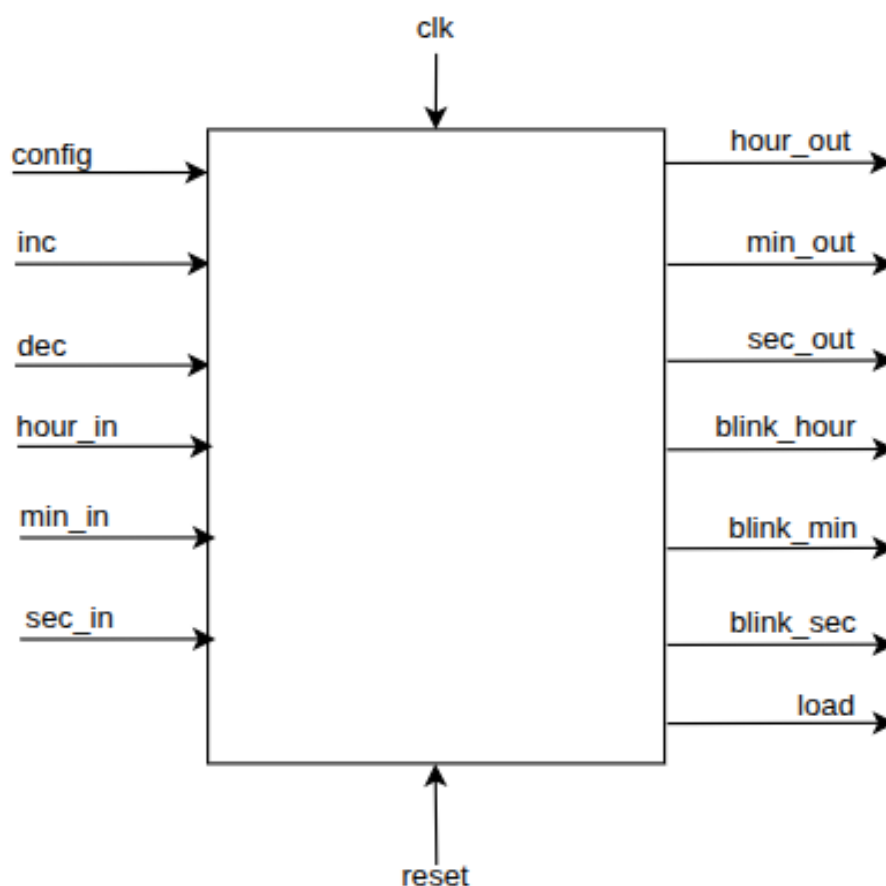


Figure 1: Entradas e Saídas.

2.1. Estrutura da FSM

A máquina de estados foi implementada utilizando os seguintes estados:

O estado IDLE representa o funcionamento normal do relógio.

Os estados SET_HOUR, SET_MIN e SET_SEC representam os modos de configuração dos respectivos campos.

Os estados INC e DEC realizam as operações de incremento e decremento dos valores configurados.

3. Processo de Desenvolvimento

O desenvolvimento foi realizado nas seguintes etapas:

- Estudo do código base fornecido;
- Análise dos requisitos da atividade;
- Construção do diagrama de estados;
- Implementação da FSM em VHDL;
- Integração da FSM ao projeto do relógio;
- Criação do ambiente de testes;
- Simulação no ModelSim;
- Compilação no Quartus;
- Testes na placa DE2-115.

Durante o desenvolvimento foram realizados ajustes nas transições da FSM e nos sinais de controle responsáveis pela carga dos novos valores configurados.

4. Diagrama da Máquina de Estados

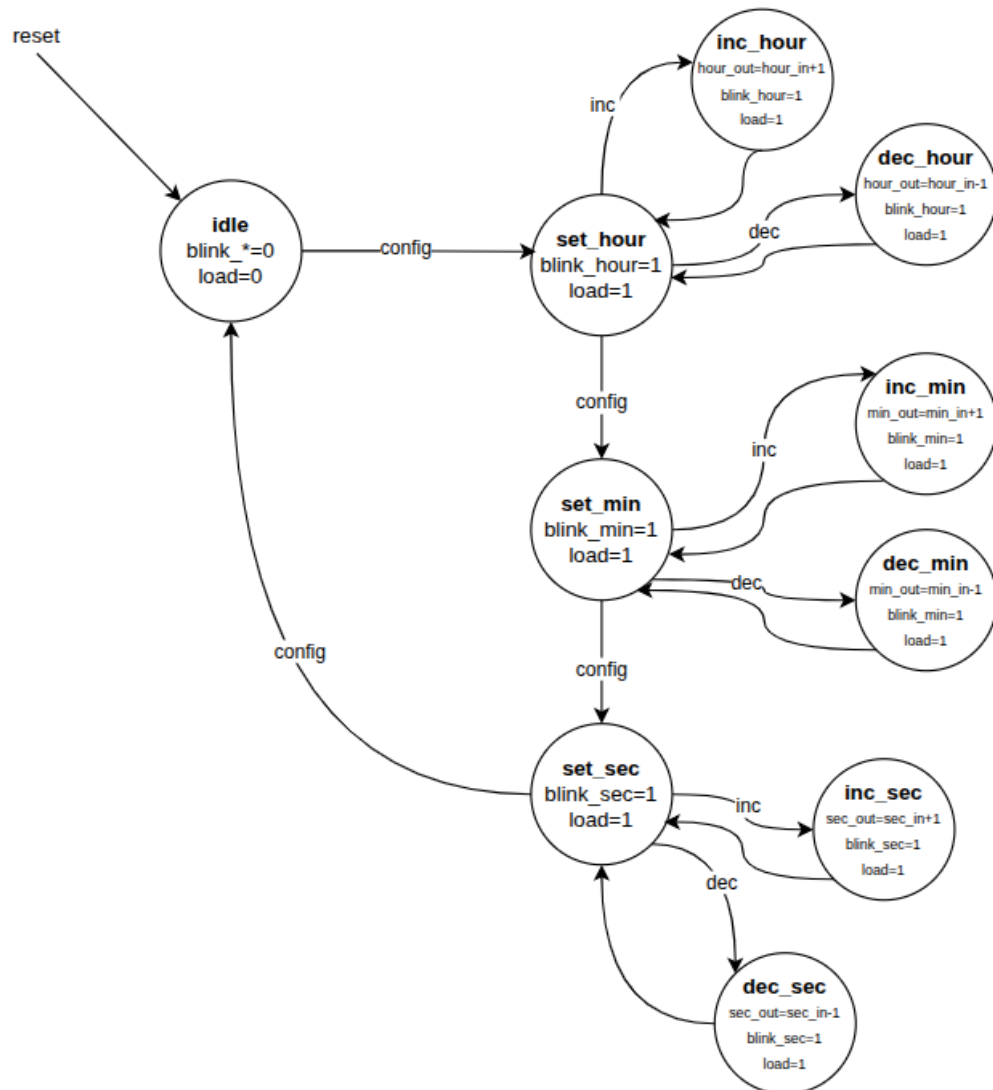


Figure 2: Diagrama da Máquina de Estados Finitos.

4.1. Implementação da FSM em VHDL

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity fsm_config_relogio is
  port(
    clk, reset : in std_logic;

    -- botoes ja tratados como ativos em '1'
    config : in std_logic;
    inc    : in std_logic;
    dec    : in std_logic;
  );
end entity;

```

```

-- valor atual vindo do timer
hour_in  : in std_logic_vector(4 downto 0);
min_in   : in std_logic_vector(5 downto 0);
sec_in   : in std_logic_vector(5 downto 0);

-- valor configurado enviado para o timer
hour_out : out std_logic_vector(4 downto 0);
min_out  : out std_logic_vector(5 downto 0);
sec_out  : out std_logic_vector(5 downto 0);

blink_hour : out std_logic;
blink_min  : out std_logic;
blink_sec  : out std_logic;
load       : out std_logic
);
end entity;

architecture rtl of fsm_config_relogio is

    type state_type is (
        IDLE,
        SET_HOUR, INC_HOUR, DEC_HOUR,
        SET_MIN,  INC_MIN,  DEC_MIN,
        SET_SEC,  INC_SEC,  DEC_SEC
    );

    signal state_reg, state_next : state_type;

    signal h_reg, h_next : unsigned(4 downto 0);
    signal m_reg, m_next : unsigned(5 downto 0);
    signal s_reg, s_next : unsigned(5 downto 0);

    -- detectores de borda: geram pulso de 1 clock por tecla pressionada
    signal config_d, inc_d, dec_d : std_logic;
    signal config_p, inc_p, dec_p : std_logic;

begin

    config_p <= config and not config_d;
    inc_p    <= inc    and not inc_d;
    dec_p    <= dec    and not dec_d;

    process(clk, reset)
    begin
        if reset = '1' then
            config_d <= '0';
            inc_d    <= '0';
            dec_d    <= '0';
        elsif rising_edge(clk) then
            config_d <= config;
            inc_d    <= inc;
            dec_d    <= dec;
        end if;
    end process;
end architecture;

```

```

    end if;
end process;

-- registradores da FSM e dos valores configurados
process(clk, reset)
begin
    if reset = '1' then
        state_reg <= IDLE;
        h_reg <= (others => '0');
        m_reg <= (others => '0');
        s_reg <= (others => '0');
    elsif rising_edge(clk) then
        state_reg <= state_next;
        h_reg <= h_next;
        m_reg <= m_next;
        s_reg <= s_next;
    end if;
end process;

-- logica de proximo estado e operacoes
process(state_reg, config_p, inc_p, dec_p, h_reg, m_reg, s_reg,
        hour_in, min_in, sec_in)
begin
    state_next <= state_reg;
    h_next <= h_reg;
    m_next <= m_reg;
    s_next <= s_reg;

    case state_reg is

        when IDLE =>
            if config_p = '1' then
                h_next <= unsigned(hour_in);
                m_next <= unsigned(min_in);
                s_next <= unsigned(sec_in);
                state_next <= SET_HOUR;
            end if;

        when SET_HOUR =>
            if config_p = '1' then
                state_next <= SET_MIN;
            elsif inc_p = '1' then
                state_next <= INC_HOUR;
            elsif dec_p = '1' then
                state_next <= DEC_HOUR;
            end if;

        when INC_HOUR =>
            if h_reg = 23 then
                h_next <= (others => '0');
            else
                h_next <= h_reg + 1;
            end if;
        end case;
    end process;

```

```

    end if;
    state_next <= SET_HOUR;

when DEC_HOUR =>
    if h_reg = 0 then
        h_next <= to_unsigned(23, 5);
    else
        h_next <= h_reg - 1;
    end if;
    state_next <= SET_HOUR;

when SET_MIN =>
    if config_p = '1' then
        state_next <= SET_SEC;
    elsif inc_p = '1' then
        state_next <= INC_MIN;
    elsif dec_p = '1' then
        state_next <= DEC_MIN;
    end if;

when INC_MIN =>
    if m_reg = 59 then
        m_next <= (others => '0');
    else
        m_next <= m_reg + 1;
    end if;
    state_next <= SET_MIN;

when DEC_MIN =>
    if m_reg = 0 then
        m_next <= to_unsigned(59, 6);
    else
        m_next <= m_reg - 1;
    end if;
    state_next <= SET_MIN;

when SET_SEC =>
    if config_p = '1' then
        state_next <= IDLE;
    elsif inc_p = '1' then
        state_next <= INC_SEC;
    elsif dec_p = '1' then
        state_next <= DEC_SEC;
    end if;

when INC_SEC =>
    if s_reg = 59 then
        s_next <= (others => '0');
    else
        s_next <= s_reg + 1;
    end if;
    state_next <= SET_SEC;

```

```

when DEC_SEC =>
    if s_reg = 0 then
        s_next <= to_unsigned(59, 6);
    else
        s_next <= s_reg - 1;
    end if;
    state_next <= SET_SEC;

end case;
end process;

-- saidas da FSM
blink_hour <= '1' when state_reg = SET_HOUR or state_reg = INC_HOUR or
state_reg = DEC_HOUR else '0';
blink_min <= '1' when state_reg = SET_MIN or state_reg = INC_MIN or
state_reg = DEC_MIN else '0';
blink_sec <= '1' when state_reg = SET_SEC or state_reg = INC_SEC or
state_reg = DEC_SEC else '0';

load <= '0' when state_reg = IDLE else '1';

hour_out <= std_logic_vector(h_reg);
min_out <= std_logic_vector(m_reg);
sec_out <= std_logic_vector(s_reg);

end architecture;

```

4.2. Modificações no arquivo top_timer_de2_115.vhd

Para integrar a máquina de estados ao projeto original, foram realizadas algumas modificações no arquivo top_timer_de2_115.vhd.

A primeira alteração foi na entrada dos botões da placa. Como a atividade utiliza quatro teclas (KEY(0) até KEY(3)), o vetor de entrada foi ajustado para quatro bits:

```
KEY : in std_logic_vector (3 downto 0);
```

Também foram adicionados sinais internos para representar as funções dos botões:

```

signal reset, load : std_logic;
signal blink_o      : std_logic;
signal blink_s, blink_m, blink_h : std_logic;
signal config, inc, dec : std_logic;
signal blink_en : std_logic;
signal k_in : std_logic_vector(2 downto 0);
signal k_out : std_logic_vector(2 downto 0);

```

Como os botões da DE2-115 são ativos em nível baixo, foi feita a inversão dos sinais:

```
-- Na DE2-115 os botoes KEY sao ativos em 0
reset <= not KEY(0);
k_in <= not KEY(1) & not KEY(2) & not KEY(3);
```

Em seguida, foi adicionada a instância da FSM de configuração do relógio. Essa FSM recebe os valores atuais de hora, minuto e segundo, além dos sinais de controle, e gera os novos valores configurados:

```
fsm1 : fsm_config_relogio
port map(
  clk => CLOCK_50,
  reset => reset,
  config => k_out(0),
  inc => k_out(1),
  dec => k_out(2),
  hour_in => hour,
  min_in => min,
  sec_in => sec,
  hour_out => hour_cfg,
  min_out => min_cfg,
  sec_out => sec_cfg,
  blink_hour => blink_h,
  blink_min => blink_m,
  blink_sec => blink_s,
  load => load
);
```

Foi adicionada a instância do sync_keys.vhd.

```
component sync_keys is
port(
  clk, reset: in std_logic;
  keys_i: in std_logic_vector(2 downto 0); -- entradas assíncronas
  keys_o: out std_logic_vector(2 downto 0) -- saídas sincronizadas
);
end component;

u_sync: sync_keys
port map(
  clk => CLOCK_50,
  reset => reset,
  keys_i => k_in,
  keys_o => k_out
);
```

A instância do temporizador também foi modificada para receber os valores vindos da FSM. Dessa forma, quando o sinal `load` é ativado, os valores configurados são carregados no relógio:

```
t0 : timer
  port map(
    clk    => CLOCK_50,
    reset  => reset,
    load   => load,
    hour_i => hour_cfg,
    sec_i  => sec_cfg,
    min_i  => min_cfg,
    hour   => hour,
    sec    => sec,
    min    => min
  );
```

Por fim, foi adicionada a lógica de pisca dos displays. O sinal `blink_en` ativa o módulo de pisca sempre que algum campo está sendo configurado:

```
blink_en <= blink_h or blink_m or blink_s;
```

Os valores enviados aos displays são apagados temporariamente quando o respectivo campo está selecionado:

```
secU_blink <= (others=>'1') when (blink_o='1' and blink_s='1') else secU;
secT_blink <= (others=>'1') when (blink_o='1' and blink_s='1') else secT;
minU_blink <= (others=>'1') when (blink_o='1' and blink_m='1') else minU;
minT_blink <= (others=>'1') when (blink_o='1' and blink_m='1') else minT;
hourU_blink <= (others=>'1') when (blink_o='1' and blink_h='1') else hourU;
hourT_blink <= (others=>'1') when (blink_o='1' and blink_h='1') else hourT;
```

Com essas alterações, o arquivo `top_timer_de2_115.vhd` passou a integrar o temporizador original, a FSM de configuração e a lógica visual de indicação do campo selecionado.

5. Testes Realizados

Para validar o funcionamento da máquina de estados, foi criado um arquivo de simulação no ModelSim utilizando comandos em um arquivo `.do`. Esse arquivo aplica estímulos nos sinais de entrada da FSM, simulando o uso dos botões da placa DE2-115.

A sequência de teste verifica o reset inicial, a entrada no modo de configuração, o incremento e decremento dos campos de hora, minuto e segundo, além do retorno ao estado IDLE.

```
# =====
# run_tb_fsm_config.do
```

```

# Teste da FSM de configuracao do relógio
# KEY0 = reset | KEY1 = config | KEY2 = inc | KEY3 = dec
# Sinais config/inc/dec aqui ja sao ativos em '1'
# =====

# Limpa simulacao anterior
quit -sim

# Cria biblioteca de trabalho
vlib work
vmap work work

# Compila apenas a FSM
vcom -2008 fsm_config_relogio.vhd

# Inicia simulacao da entidade da FSM
vsim work.fsm_config_relogio

# Adiciona sinais na forma de onda
add wave -divider "CLOCK E RESET"
add wave -radix binary sim:/fsm_config_relogio/clk
add wave -radix binary sim:/fsm_config_relogio/reset

add wave -divider "ENTRADAS - BOTOES"
add wave -radix binary sim:/fsm_config_relogio/config
add wave -radix binary sim:/fsm_config_relogio/inc
add wave -radix binary sim:/fsm_config_relogio/dec

add wave -divider "ENTRADAS - VALOR ATUAL DO TIMER"
add wave -radix unsigned sim:/fsm_config_relogio/hour_in
add wave -radix unsigned sim:/fsm_config_relogio/min_in
add wave -radix unsigned sim:/fsm_config_relogio/sec_in

add wave -divider "ESTADOS INTERNOS"
add wave sim:/fsm_config_relogio/state_reg
add wave sim:/fsm_config_relogio/state_next

add wave -divider "SAIDAS - VALORES CONFIGURADOS"
add wave -radix unsigned sim:/fsm_config_relogio/hour_out
add wave -radix unsigned sim:/fsm_config_relogio/min_out
add wave -radix unsigned sim:/fsm_config_relogio/sec_out

add wave -divider "SAIDAS DE CONTROLE"
add wave -radix binary sim:/fsm_config_relogio/blink_hour
add wave -radix binary sim:/fsm_config_relogio/blink_min
add wave -radix binary sim:/fsm_config_relogio/blink_sec
add wave -radix binary sim:/fsm_config_relogio/load

# Clock de 10 ns
force -freeze sim:/fsm_config_relogio/clk 0 0 ns, 1 5 ns -repeat 10 ns

# Valores iniciais

```

```

force sim:/fsm_config_relogio/reset 1
force sim:/fsm_config_relogio/config 0
force sim:/fsm_config_relogio/inc 0
force sim:/fsm_config_relogio/dec 0

```

```

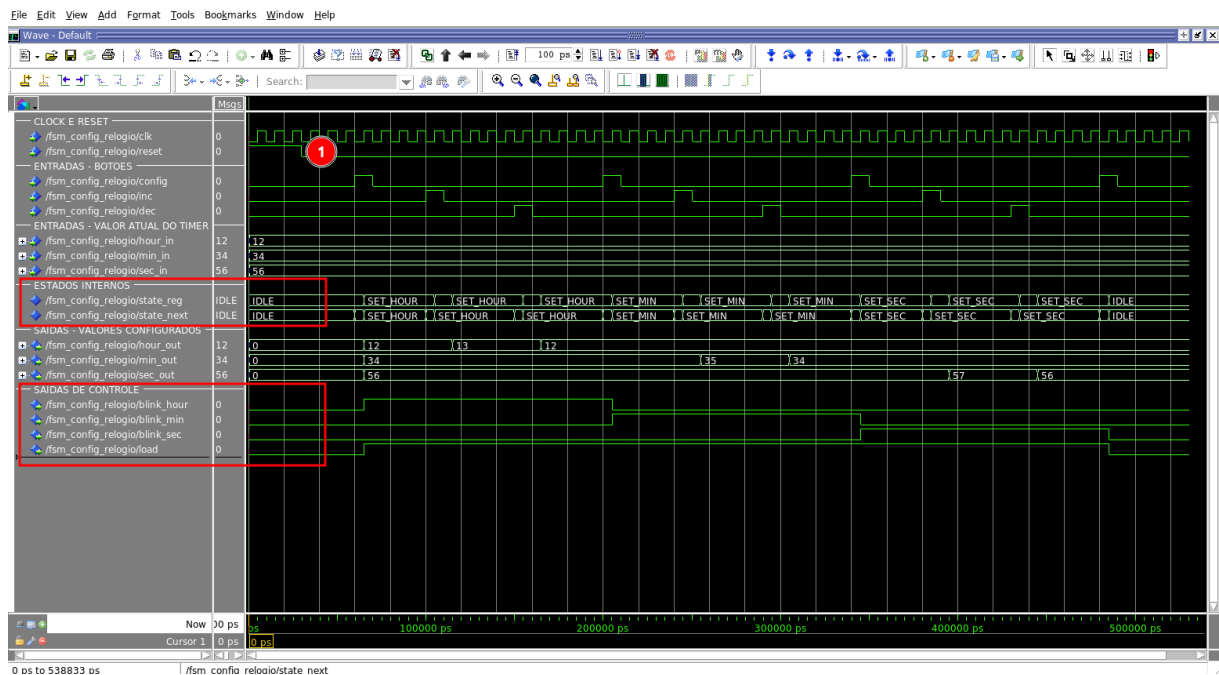
# Relogio inicialmente em 12:34:56
force sim:/fsm_config_relogio/hour_in 5'd12
force sim:/fsm_config_relogio/min_in 6'd34
force sim:/fsm_config_relogio/sec_in 6'd56

```

```

# =====
# TESTE 1 - RESET
# Esperado: estado IDLE, load=0, blink_* = 0
# =====
run 30 ns
force sim:/fsm_config_relogio/reset 0
run 30 ns

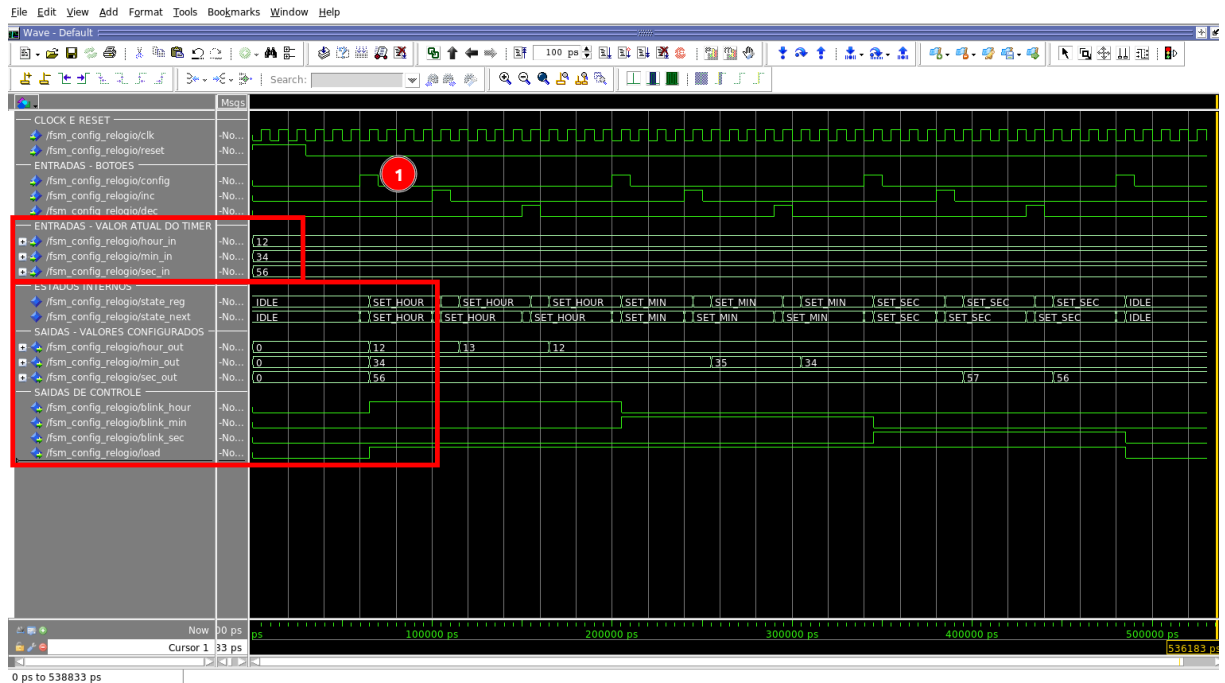
```



```

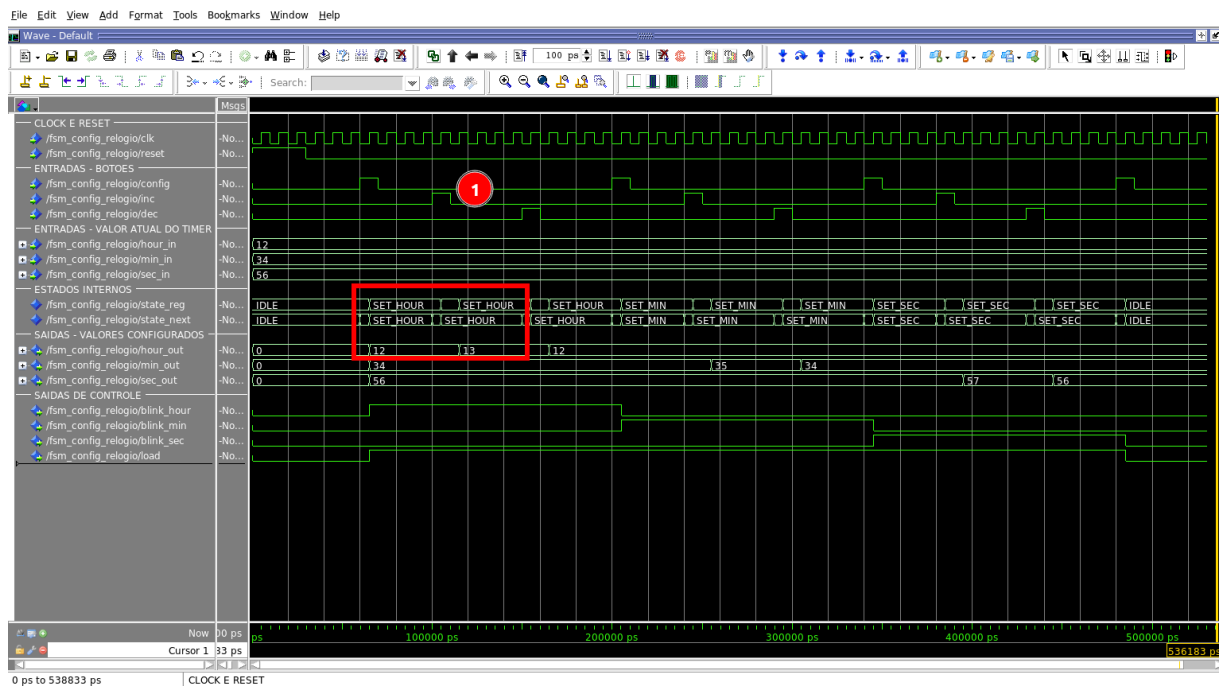
# =====
# TESTE 2 - KEY1/config: IDLE -> SET_HOUR
# Esperado: carrega 12:34:56 nos registradores internos,
# blink_hour=1, load=1
# =====
force sim:/fsm_config_relogio/config 1
run 10 ns
force sim:/fsm_config_relogio/config 0
run 30 ns

```



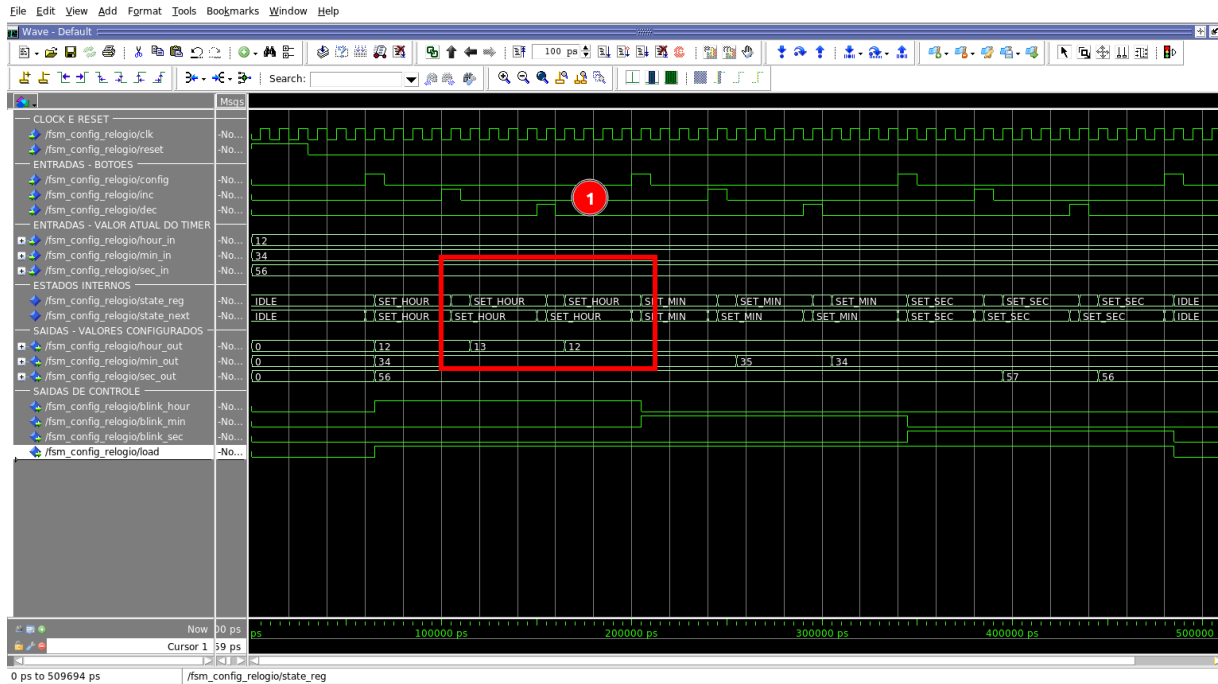
```
# =====
# TESTE 3 - KEY2/inc em SET_HOUR
# Esperado: SET_HOUR -> INC_HOUR -> SET_HOUR
# hora 12 -> 13
# =====

force sim:/fsm_config_relogio/inc 1
run 10 ns
force sim:/fsm_config_relogio/inc 0
run 40 ns
```



```
# =====
# TESTE 4 - KEY3/dec em SET_HOUR
# Esperado: SET_HOUR -> DEC_HOUR -> SET_HOUR
# hora 13 -> 12
# =====

force sim:/fsm_config_relogio/dec 1
run 10 ns
force sim:/fsm_config_relogio/dec 0
run 40 ns
```

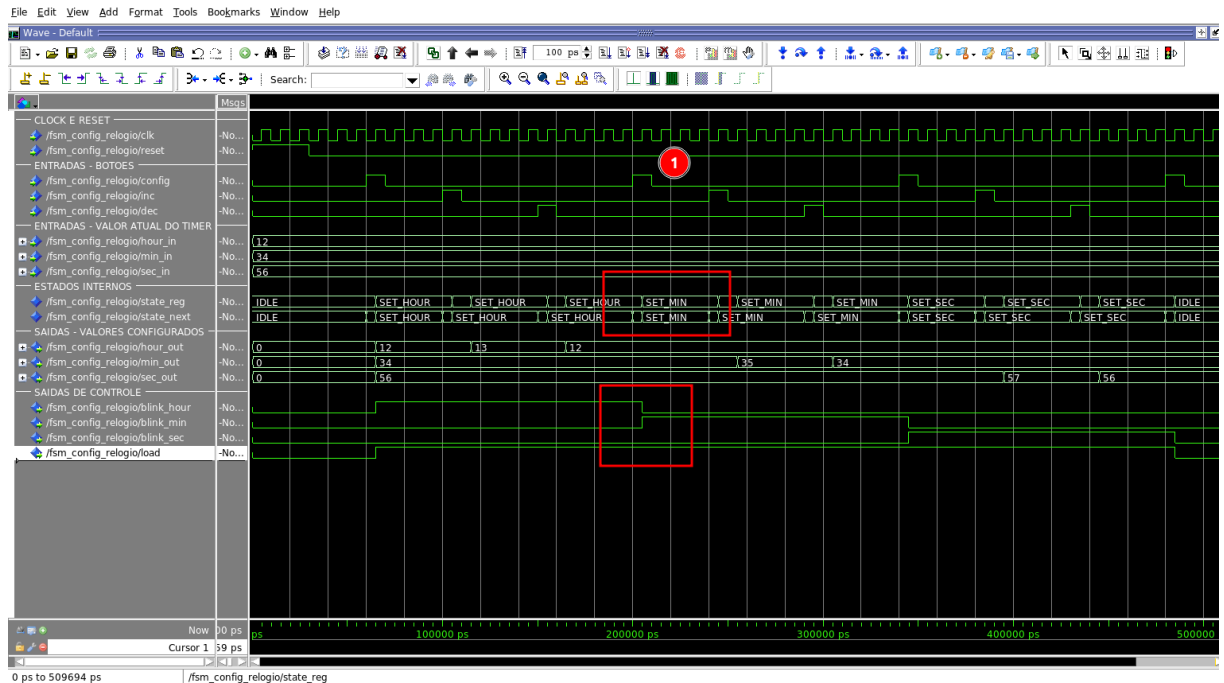


```
# =====
# TESTE 5 - KEY1/config: SET_HOUR -> SET_MIN
# Esperado: blink_min=1
# =====

force sim:/fsm_config_relogio/config 1
run 10 ns
force sim:/fsm_config_relogio/config 0
run 30 ns

# Incrementa minuto: 34 -> 35
force sim:/fsm_config_relogio/inc 1
run 10 ns
force sim:/fsm_config_relogio/inc 0
run 40 ns

# Decrementa minuto: 35 -> 34
force sim:/fsm_config_relogio/dec 1
run 10 ns
force sim:/fsm_config_relogio/dec 0
run 40 ns
```

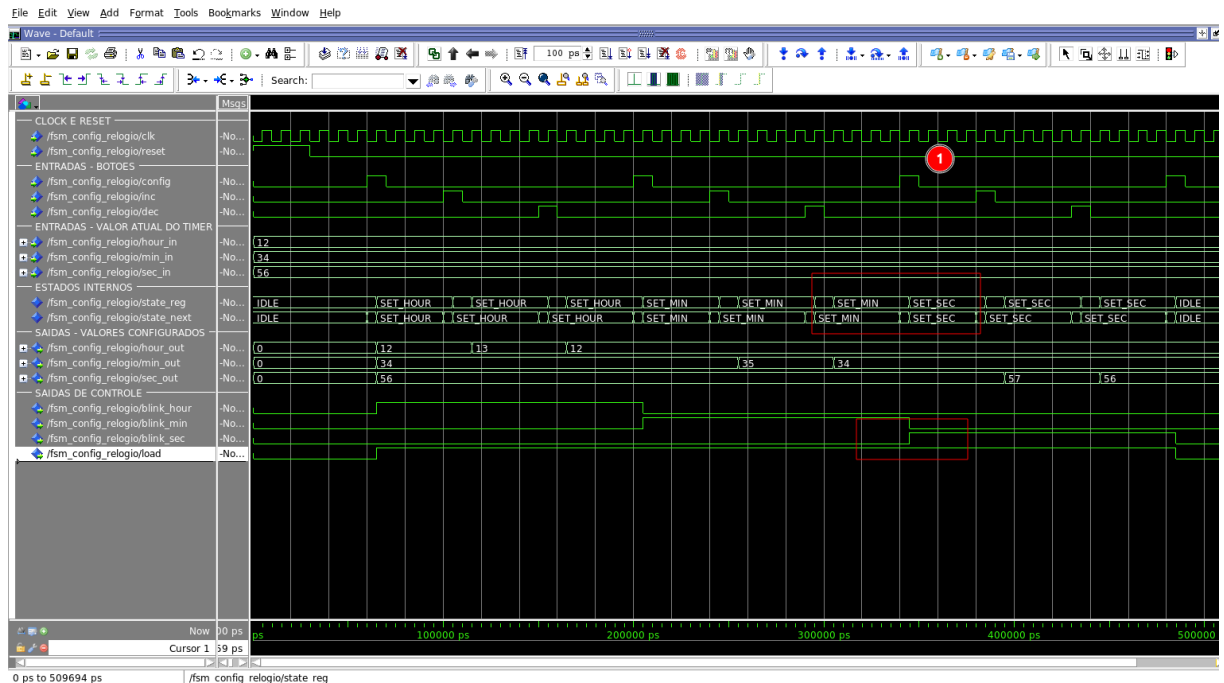


```
# =====
# TESTE 6 - KEY1/config: SET_MIN -> SET_SEC
# Esperado: blink_sec=1
# =====

force sim:/fsm_config_relogio/config 1
run 10 ns
force sim:/fsm_config_relogio/config 0
run 30 ns

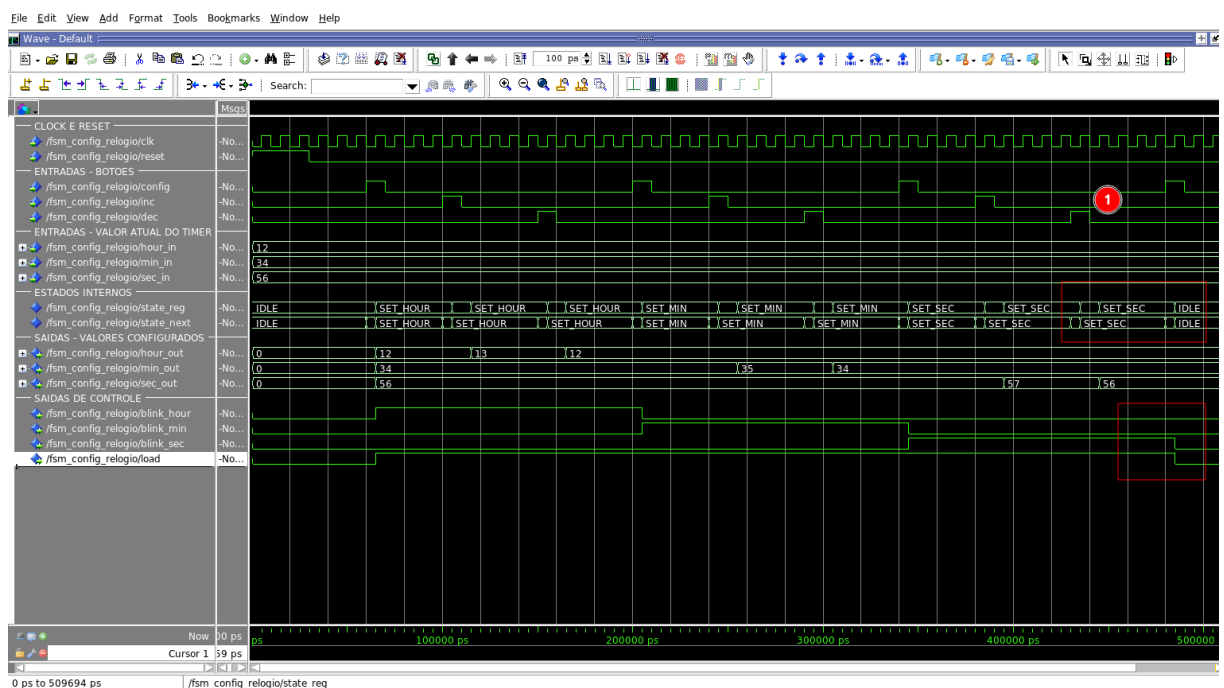
# Incrementa segundo: 56 -> 57
force sim:/fsm_config_relogio/inc 1
run 10 ns
force sim:/fsm_config_relogio/inc 0
run 40 ns

# Decrementa segundo: 57 -> 56
force sim:/fsm_config_relogio/dec 1
run 10 ns
force sim:/fsm_config_relogio/dec 0
run 40 ns
```

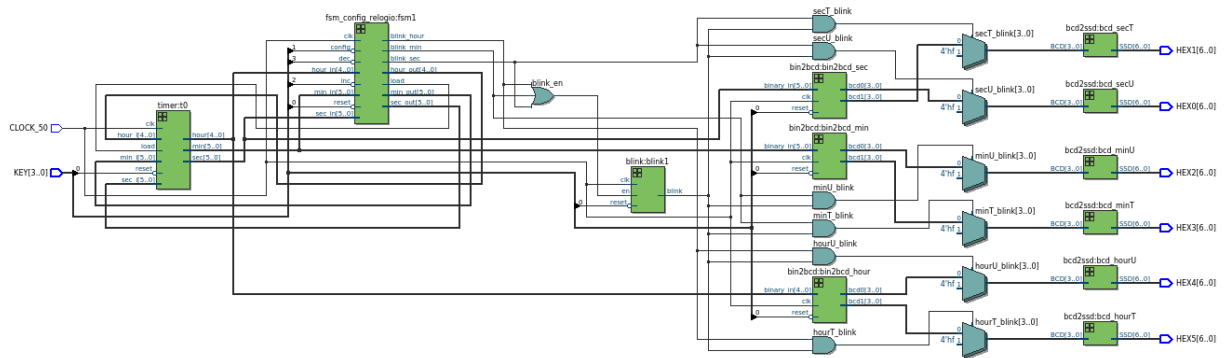


```
# =====
# TESTE 7 - KEY1/config: SET_SEC -> IDLE
# Esperado: estado IDLE, load=0, blink_* = 0
# =====
force sim:/fsm_config_relogio/config 1
run 10 ns
force sim:/fsm_config_relogio/config 0
run 40 ns

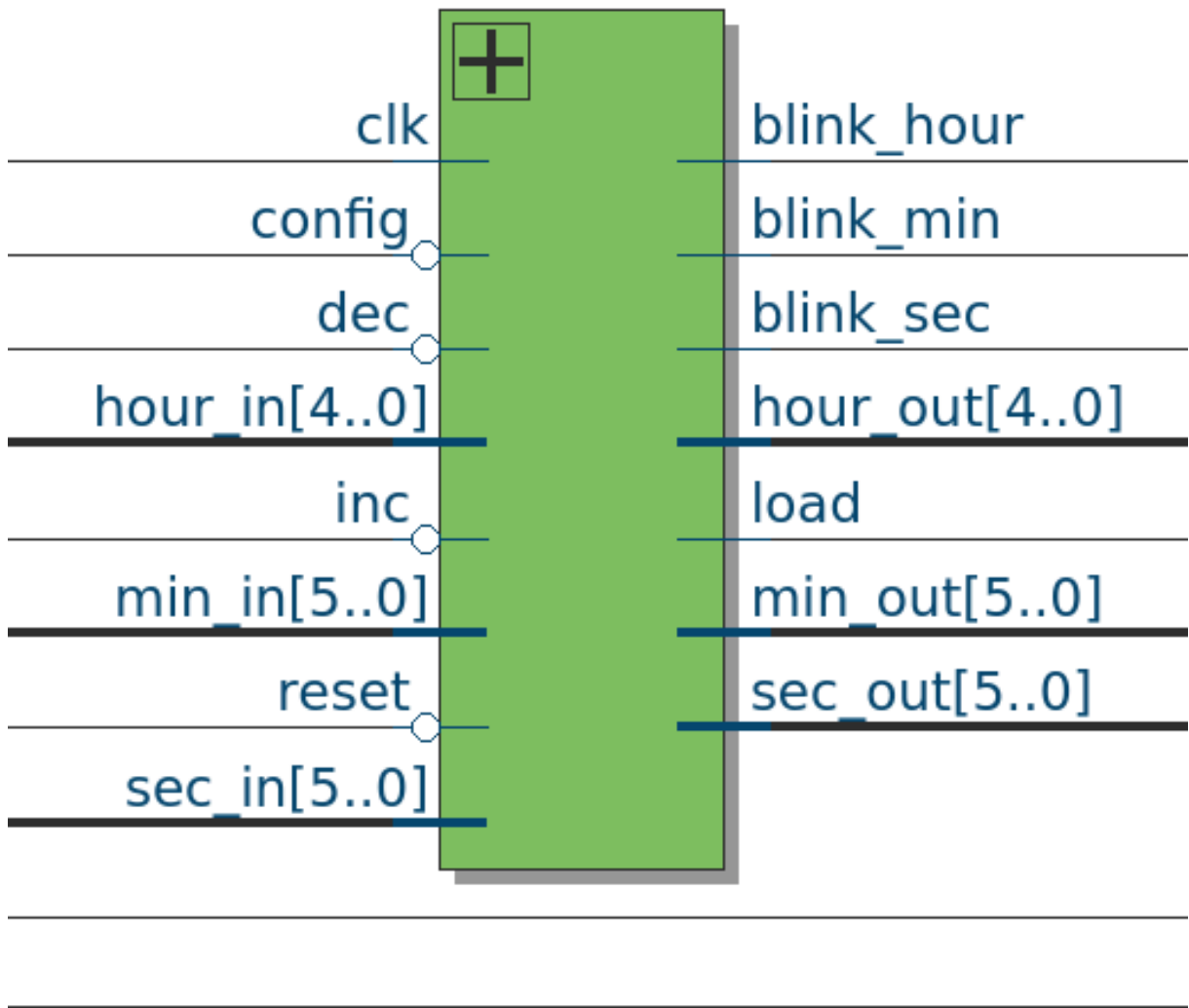
# Ajusta visualizacao
wave zoom full
```



6. RTL Viewer



fsm_config_relogio:fsm1



7. Conclusão

Neste trabalho foi desenvolvida uma Máquina de Estados Finitos (FSM) para o controle da configuração de um relógio digital implementado em VHDL para a plataforma DE2-115. A solução proposta permitiu a navegação entre os campos de hora, minuto e segundo através do sinal de configuração, bem como a alteração dos respectivos valores por meio dos sinais de incremento e decremento.

Durante o desenvolvimento, foi realizada a integração da FSM ao projeto original do relógio, incluindo a implementação dos sinais de controle, do mecanismo de carga dos novos valores configurados e da indicação visual do campo selecionado através dos sinais de pisca dos displays.

A validação foi realizada por meio de simulações no ModelSim, nas quais foi possível verificar corretamente as transições entre os estados da máquina, o funcionamento dos comandos de incremento e decremento e o retorno ao estado de operação normal após a finalização da configuração. Além disso, o projeto foi compilado com sucesso no Quartus Prime, sem a ocorrência de erros de síntese ou implementação.

A atividade permitiu consolidar os conceitos de projeto síncrono e de máquinas de estados estudados na disciplina, demonstrando a aplicação prática desses conceitos no desenvolvimento de sistemas digitais utilizando VHDL e FPGA.